Enumerate, break, continue, built in function

◆ 1. `enumerate()` কী?

Python এর **`enumerate()`** function ব্যবহার করলে তুমি **list বা iterable এর index + value** একসাথে পেতে পারো।

Syntax:

```
enumerate(iterable, start=0)
```

- `iterable` → list, tuple, string ইত্যাদি
- `start` → index কোথা থেকে শুরু হবে (default 0)

◆ 2. সাধারণ Example

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output:

```
0 apple
1 banana
2 cherry
```

☞ এখানে আমরা একই লুপে **index এবং value দুটোই** পেয়েছি।

◆ 3. Index শুরু অন্য থেকে করা

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits, start=1):
    print(index, fruit)
```

Output:

```
1 apple
2 banana
3 cherry
```

◆ 4. Unpacking দিয়ে ব্যবহার করা

`enumerate()` return করে **tuple** `(index, value)`। তাই তুমি **unpack** করতে পারো:

```
fruits = ["apple", "banana", "cherry"]

for item in enumerate(fruits):
    print(item)  # tuple হিসেবে
```

Output:

```
(0, 'apple')
(1, 'banana')
(2, 'cherry')
```

এখানে `item` হল tuple।
আর যদি unpack করতে চাও:

```
for idx, val in enumerate(fruits):
    print("Index:", idx, "Value:", val)
```

◆ 5. Example with tuple unpacking

```
data = [("A", 10), ("B", 20), ("C", 30)]

for index, (letter, number) in enumerate(data):
    print(index, letter, number)
```

Output:

```
0 A 10
1 B 20
2 C 30
```

☞ এখানে আমরা **nested unpacking** করেছি – `enumerate` এর tuple + ভিতরের tuple।

◆ সংক্ষেপে:

- enumerate() → index + value দেয়
- unpacking → tuple থেকে আলাদা variable এ assign করা
- nested structure → (index, (a, b)) → index, (a, b) unpack করা যায়

◈ break কী?

break হলো **loop control statement**।
যখন break চালানো হয়, তখন **loop সাথে সাথেই থেমে যায়** (আর চলতে থাকে না) এবং control লুপের বাইরে চলে যায়।

◈ Example 1: For loop এ break

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```

☞ এখানে i == 5 হলে loop থেমে গেছে।

◈ Example 2: While loop এ break

```
n = 1
while n <= 10:
    if n == 6:
        break
    print(n)
    n += 1
```

Output:

```
1
2
```

```
3
4
5
```

☞ যখন `n == 6`, তখন `break` loop বন্ধ করে দিয়েছে।

◈ Example 3: `break` দিয়ে search করা

```
numbers = [10, 20, 30, 40, 50]

for num in numbers:
    if num == 30:
        print("Found:", num)
        break
```

Output:

```
Found: 30
```

☞ এখানে **প্রথম match** পেয়ে loop থেমে গেছে।

◈ Important Note

- `break` শুধু **এক লুপ** থামায়। যদি nested loop (লুপের ভেতরে লুপ) থাকে, তবে শুধু ভেতরের লুপ থামবে।

◈ Example: Nested Loop with `break`

```
for i in range(3):              # outer loop
    for j in range(5):          # inner loop
        if j == 3:
            break               # শুধু inner loop থামবে
        print(f"i={i}, j={j}")
```

Output:

```
i=0, j=0
i=0, j=1
i=0, j=2
i=1, j=0
i=1, j=1
i=1, j=2
i=2, j=0
i=2, j=1
i=2, j=2
```

☞ এখানে `j == 3` হলে **ভেতরের loop থেমে গেছে**।
কিন্তু outer loop (`i loop`) চলতে থেকেছে।

◈ Nested Loop সম্পূর্ণ ভাঙতে চাইলে?

যদি outer loop-ও ভাঙতে চাও, তখন একটা **flag variable** বা **function return** ব্যবহার করতে হয়।

Example with Flag:

```
found = False

for i in range(3):
    for j in range(5):
        if j == 3:
            found = True
            break   # inner loop break
        print(f"i={i}, j={j}")
    if found:
        break       # outer loop break

print("Loop stopped")
```

☞ এখানে `j == 3` হলে inner loop ভাঙে, তারপর `found=True` দেখে outer loop-ও ভেঙে যায়।

◈ সংক্ষেপে:

- `break` শুধু **একটা loop** ভাঙে।
- Nested loop এ outer লুপ থামাতে চাইলে **flag** বা **function return** দরকার হয়।

◈ Example: 2D Matrix এ Search + Break

```
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]

target = 50
found = False   # flag

for row in range(len(matrix)):          # outer loop → row
```

```
    for col in range(len(matrix[row])):  # inner loop → column
        if matrix[row][col] == target:
            print(f"Found {target} at ({row}, {col})")
            found = True
            break   # inner loop break
    if found:
        break       # outer loop break

if not found:
    print("Not Found!")
```

🔹 Output

```
Found 50 at (1, 1)
```

👉 এখানে **50 পাওয়া মাত্র inner loop ভেঙেছে** → তারপর flag দেখে outer loop-ও ভেঙে গেছে।
তাই পুরো matrix scan না করেই search শেষ।

✅ এই টেকনিক বড় dataset (যেমন database search, game grid, বা বড় টেক্সট ফাইল) এ খুব কাজে লাগে।

🔹 continue কী?

- continue লুপের ভেতরে ব্যবহার হয়।
- যখন continue চালানো হয়, তখন **সেই iteration এর বাকি কোড skip হয়ে যায়** এবং লুপ আবার **পরের iteration এ চলে যায়**।

🔹 Example 1: For loop এ continue

```
for i in range(6):
    if i == 3:
        continue
    print(i)
```

Output:

```
0
1
2
4
5
```

👉 যখন `i == 3`, তখন `continue` হয়ে গেছে → তাই `print(i)` skip করেছে, এরপর পরের iteration (4) এ চলে গেছে।

🔹 Example 2: While loop এ continue

```
n = 0
while n < 6:
    n += 1
    if n % 2 == 0:   # even number skip
        continue
    print(n)
```

Output:

```
1
3
5
```

👉 এখানে সব even number skip করেছে, শুধু odd number print হয়েছে।

🔹 Example 3: String থেকে vowel বাদ দেওয়া

```
text = "programming"

for ch in text:
    if ch in "aeiou":
        continue
    print(ch, end="")
```

Output:

```
prgrmmng
```

👉 সব vowel skip হয়ে গেছে।

🔹 Summary

- `break` → পুরো loop ভাঙে
- `continue` → শুধু বর্তমান iteration skip করে, loop চালু থাকে

◆ Commonly Used Built-in Python Functions

1. Input / Output

- `print()` → আউটপুট দেখাতে
- `input()` → ইউজারের কাছ থেকে ইনপুট নিতে

---

2. Type Conversion

- `int()` → সংখ্যা integer বানাতে
- `float()` → সংখ্যা float বানাতে
- `str()` → string বানাতে
- `list()`, `tuple()`, `set()`, `dict()` → data type convert করতে

---

3. Length and Iteration

- `len()` → কোনো sequence এর length
- `range()` → loop এর জন্য range বানাতে
- `enumerate()` → index + value একসাথে পেতে
- `zip()` → একাধিক iterable কে pair করে দিতে

---

4. Math Related

- `abs()` → absolute value
- `min()`, `max()` → ছোটো / বড়ো value খুঁজতে
- `sum()` → যোগফল বের করতে
- `round()` → সংখ্যা রাউন্ড করতে
- `pow(x, y)` → power (x^y)

---

5. Logical / Checking

- `all()` → সব value True কিনা

- any() → অন্তত একটি True আছে কিনা
- isinstance(obj, type) → object এর type check করতে

---

6. Sorting and Reversing

- sorted() → iterable কে sort করা (নতুন list দেয়)
- reversed() → iterable কে উল্টানো

---

7. Help and Debugging

- type() → data type দেখায়
- id() → object এর unique id (memory address)
- dir() → object এর attribute/method list
- help() → কোনো function/module এর ডকুমেন্টেশন

---

8. Functional Programming

- map() → function apply করা
- filter() → শর্ত মেনে elements বেছে আনা
- reduce() → একাধিক value কে একটায় নামানো (functools থেকে import করতে হয়)
- lambda → ছোট anonymous function

---

9. File Related

- open() → ফাইল read/write করার জন্য

---

10. Others

- eval() → string কে Python expression হিসেবে evaluate করা
- sorted() → কোনো sequence কে sort করা
- hash() → object এর hash value
- chr(), ord() → character ↔ ASCII code কনভার্ট

✅ এগুলো হলো **সবচেয়ে বেশি ব্যবহৃত** **built-in Python functions**।